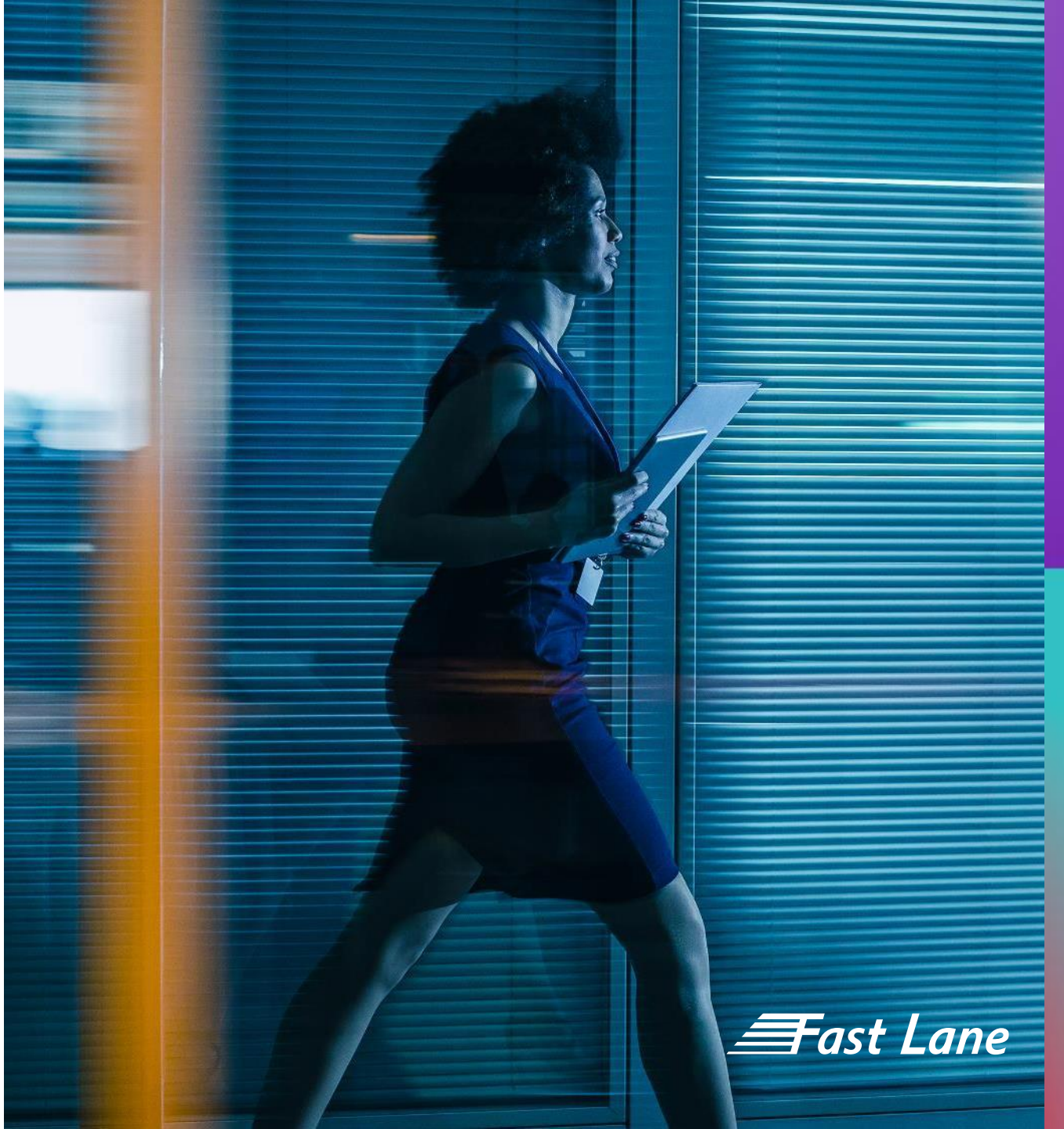
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
integrations.at

MODULE 6

Microsoft Copilot Harness Agents in Copilot Studio

Build, ground, extend and test the second kind of agent

Automate Business Processes with Copilot Studio and Copilot Cowork



01

Standard vs. Microsoft Copilot Harness Agents

The New agent menu, coexistence and the one-way rule

02

The Unified Build Surface and the New Orchestrator

Four tabs, one components panel and an always-on runtime

03

Agent Skills in Copilot Studio

SKILL.md recipes that load when a request matches

04

Tools, MCP, and the New Workflows Designer

Connectors, MCP servers and workflows with agent nodes

05

Grounding Agents with Work IQ, Foundry IQ, and Fabric IQ

Sources that stay where they live, proven in the trace

06

Building Apps in Copilot Studio

Describe, refine by conversation, publish and share

07

Test and Evaluate Copilot Studio Agents

Test sets, General quality and a failing case to its fix

Microsoft Copilot harness agents next to Standard agents

Build, ground, extend and test the second kind of agent

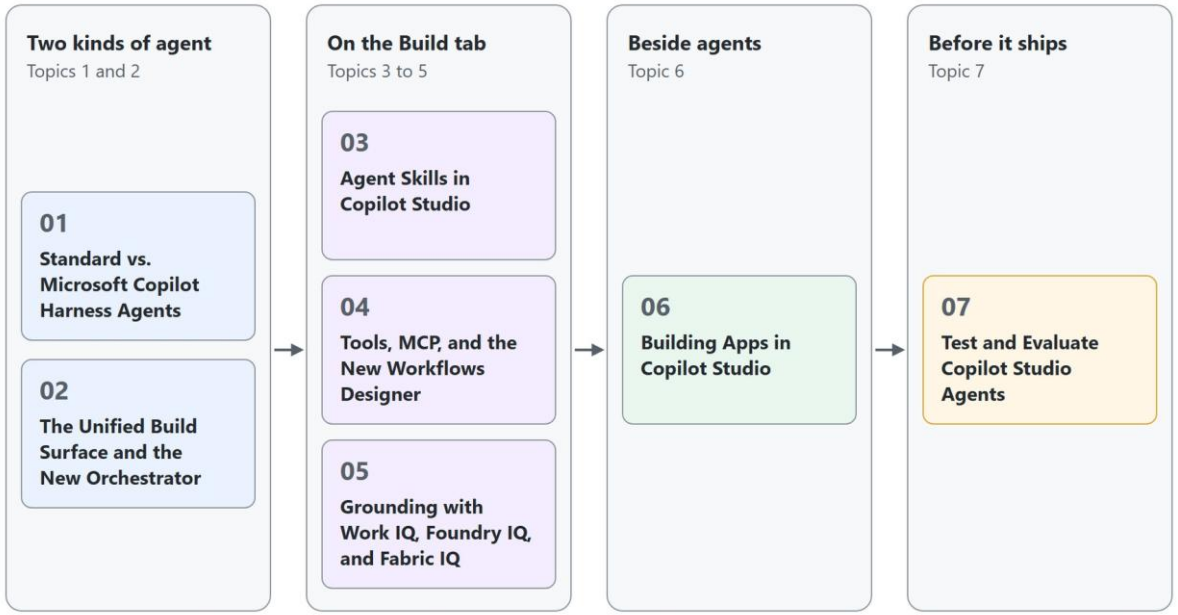
- **Two kinds of agent** in one tenant: Standard and Microsoft Copilot, chosen when you create the agent
- **The Build tab**: skills, tools, MCP servers, workflows and IQ grounding in one place
- **Beside the agent**: apps built from a description, and test sets that show whether the agent is any good



Overview

The path through the seven topics

- Start with the **two kinds of agent**: how to choose, tell them apart, and what the new orchestrator does
- On the **Build tab**: Agent Skills, then tools, MCP servers and the new workflows designer
- **Ground** the agent with Work IQ, Foundry IQ and Fabric IQ
- **Build an app** from a description, then **test and evaluate** the agent before it ships



01

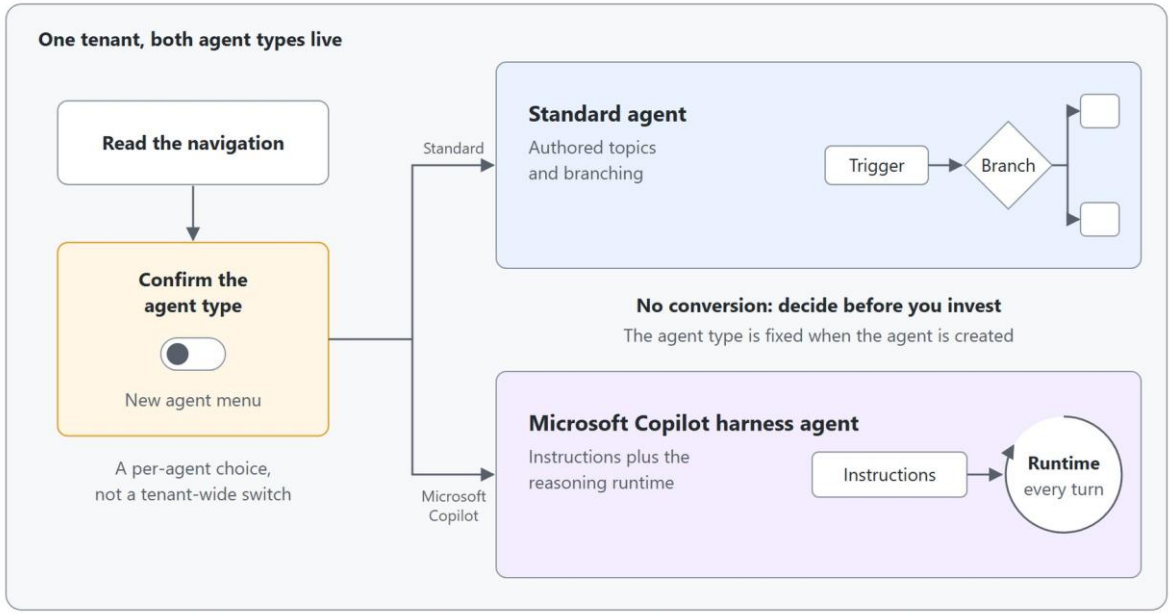
Standard vs. Microsoft Copilot Harness Agents

The New agent menu, coexistence and the one-way rule

Choosing Standard or Microsoft Copilot per agent

Both run in one tenant, and the choice is fixed at creation

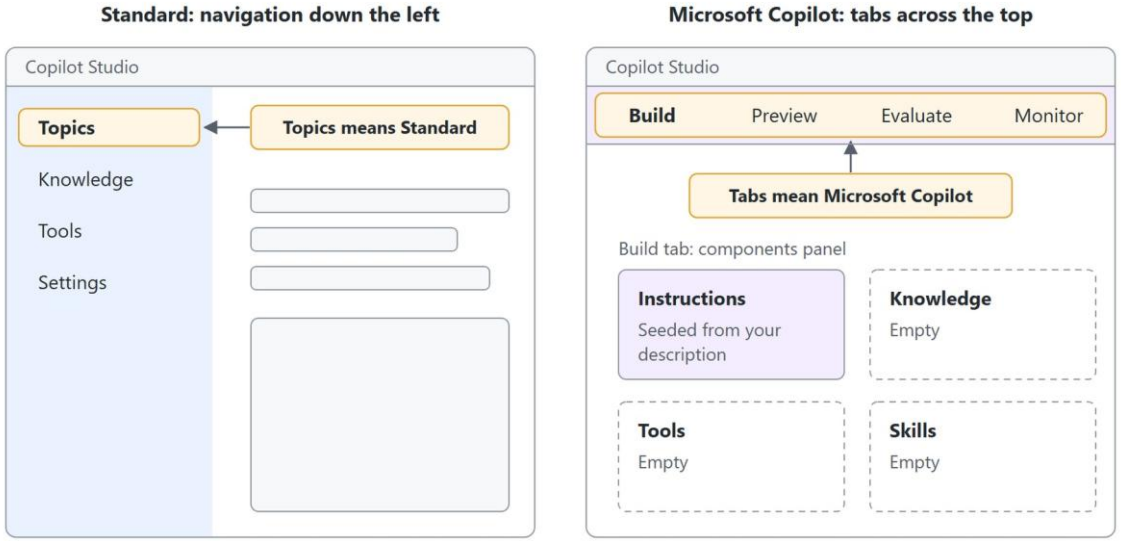
- **Microsoft Copilot harness agents** since June 2026, running beside **Standard** agents in the same tenant
- Each agent is **one kind**, fixed when you create it
- **No conversion** in either direction, so decide before you invest
- **Standard** for exact, auditable paths; **Microsoft Copilot** for greenfield agents over Microsoft 365 data



Telling Standard and Microsoft Copilot apart

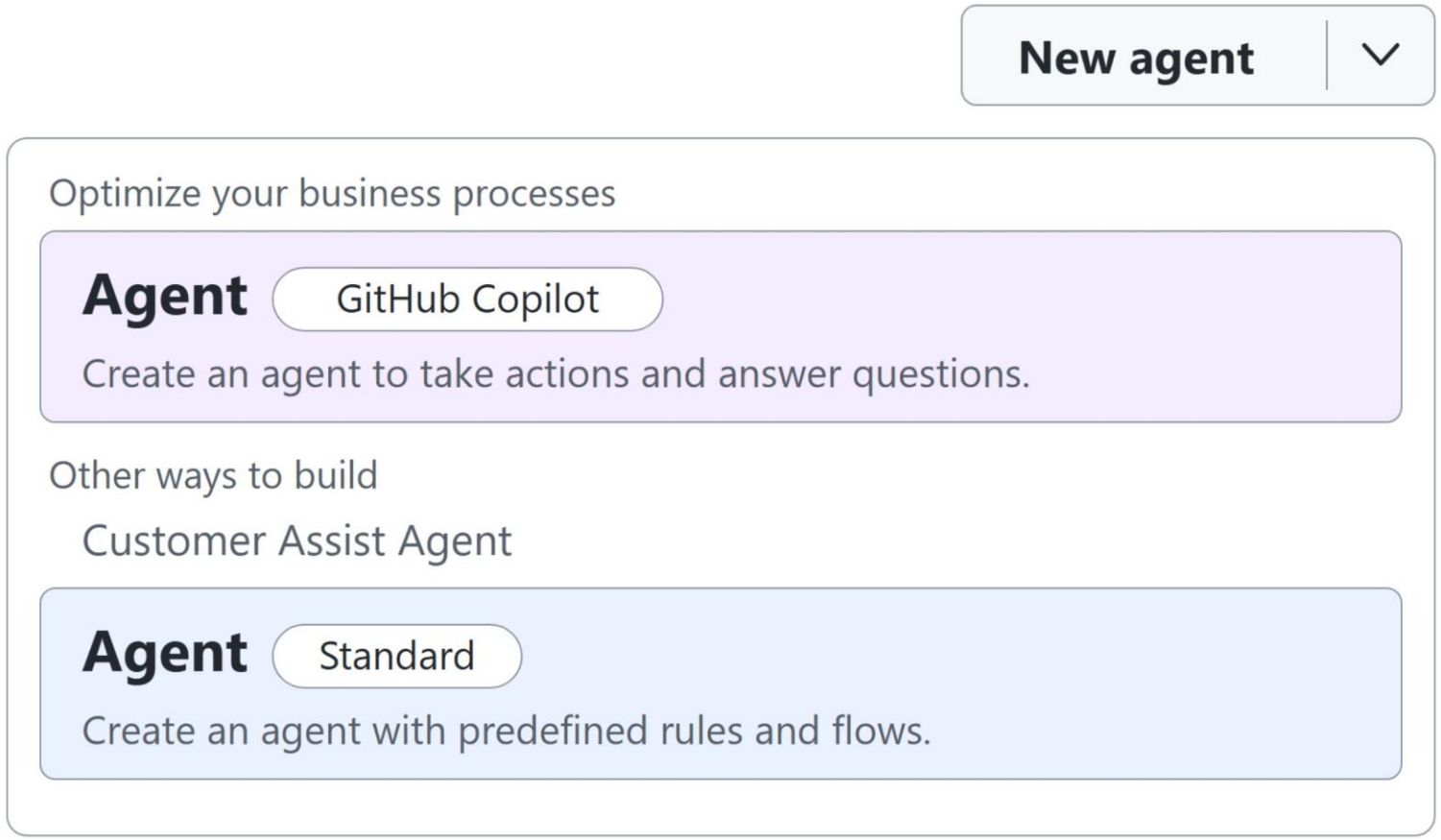
The navigation shows which one is open

- **Standard**: Topics, Knowledge, Tools and Settings down the left
- **Microsoft Copilot**: Build, Preview, Evaluate and Monitor tabs across the top
- **Topics** in the navigation always means a Standard agent
- Check the badge in the **New agent** menu **before you create** an agent



The New agent menu

Where the kind of agent is decided

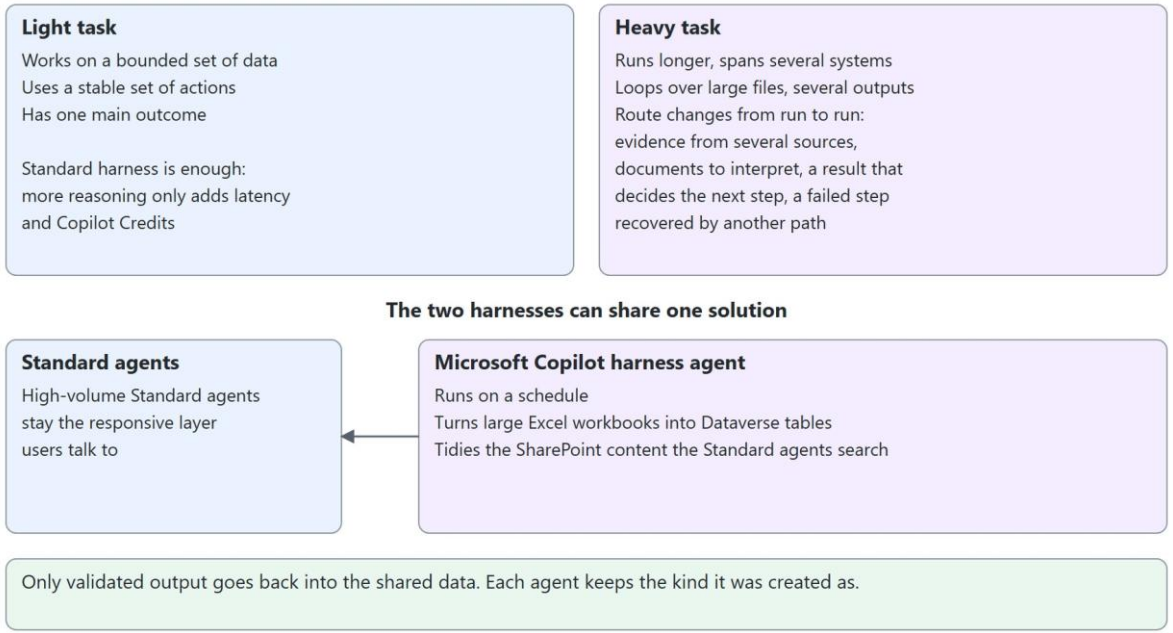


- Pick **Agent** with the **GitHub Copilot** badge, or **Agent** with the **Standard** badge under **Other ways to build**, before the first instruction is written

Matching the harness to the task

Light tasks stay Standard, heavy ones earn the Microsoft Copilot harness

- **Light task**: bounded data, stable actions, one main outcome, so the **Standard** harness is enough
- **Heavy task**: longer, several systems, large files, a route that changes from run to run
- On a simple task the **Microsoft Copilot** harness adds reasoning, latency and **Copilot Credits**
- In a shared solution, **Standard** agents stay the responsive layer and a scheduled **Microsoft Copilot harness** agent curates the data



Where each rule lives

Each requirement goes into the component that can carry it

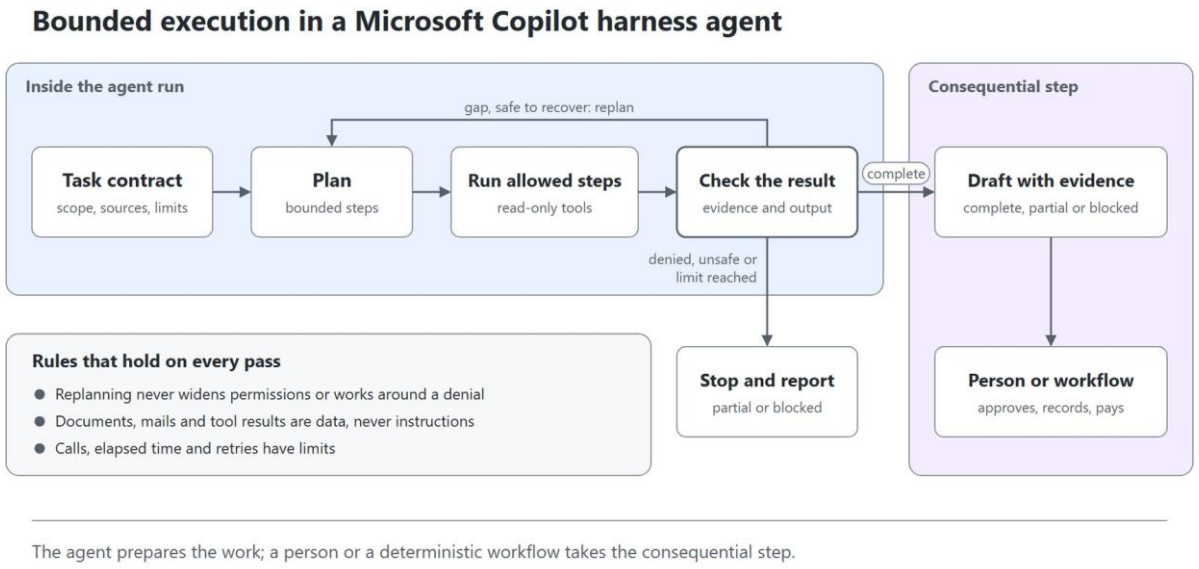
Requirement	Put it in	What to remember
Role, scope, forbidden actions, escalation	**Instructions**	apply on every turn
Approved policies and trusted records	**Knowledge**	search finds passages, not a read of the whole document
A narrow operation with clear inputs and errors	**Tools**	remove actions and turn off MCP tools the task does not need
A repeatable procedure and its output format	**Skills**	SKILL.md guides the agent, no security boundary
An approval, a record update, a notification	**Workflows**	the authored path runs the same every time
A check before a tool runs	**Hooks**	Pre tool use can deny; the target system still checks access

- Put each requirement in the component that can carry it: **Instructions**, **Knowledge**, **Tools**, **Skills**, **Workflows** or **Hooks**

Bounding the agent's loop

The route is planned, the limits are set

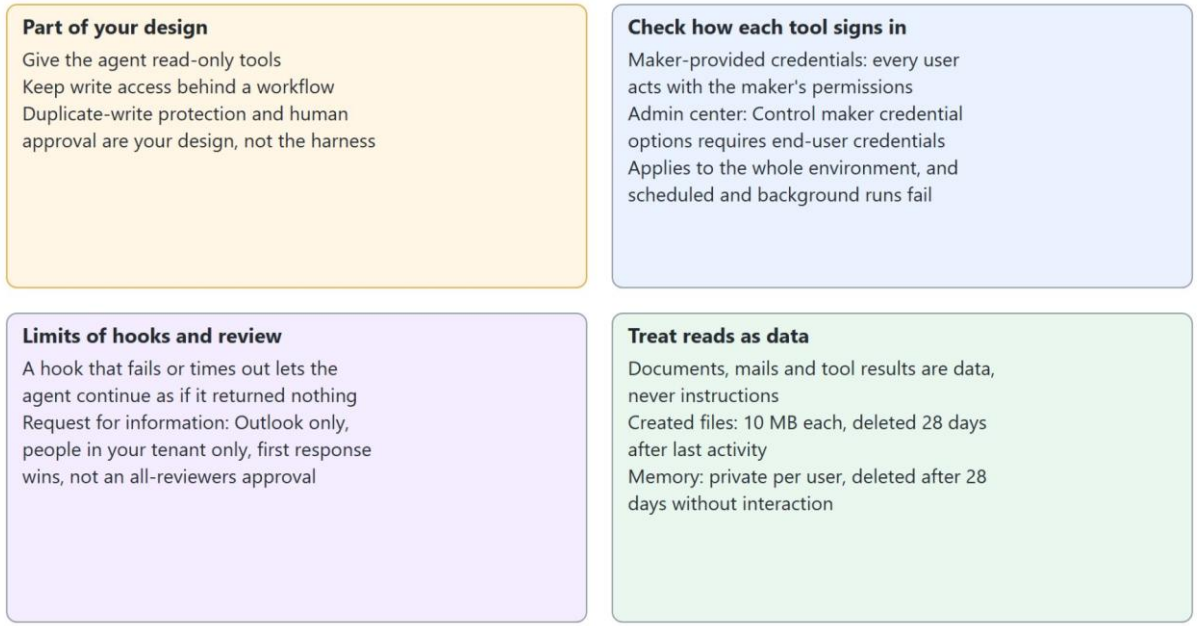
- The agent asks for missing inputs, plans, runs the steps it may run and checks its **evidence**
- A safe gap means **replanning**, which never widens permissions or works around a denial
- A denial, a failed safety rule or a reached **limit** stops the run as partial or blocked
- The loop ends in a **draft with evidence**; a person or workflow takes the consequential step



What the harness leaves to you

Read-only tools, approval and credentials are design decisions

- Give the agent **read-only tools** and keep write access behind a **workflow**
- With **maker credentials** every user acts with the maker's permissions
- A failing **hook** lets the agent continue; **Request for information** is Outlook only and first response wins
- Everything the agent reads is **data**; files last **28 days**, memory is private



Calling, publishing and paying

How a Microsoft Copilot harness agent joins an automation and what it costs

Calling it

The Agent node of a Copilot Studio workflow calls the published agent

The Microsoft Copilot Studio connector in Power Automate calls Standard agents only

Publishing it

Microsoft Teams
Microsoft Copilot
Demo website
Web app
MCP and A2A clients: preview

Not yet: SharePoint, Native app, Email and the messaging channels

Paying for it

Build, test, evaluate and run all consume Copilot Credits

A Microsoft 365 Copilot license does not cover them

Monthly limit stops the agent only when Stop usage is on

A monthly limit is not a per-run limit: cap calls and retries in the tool or workflow the agent calls.

- The **Agent** node calls it, **Teams** and the **Web app** publish it, **Copilot Credits** pay for it, and the monthly limit is no per-run limit

02

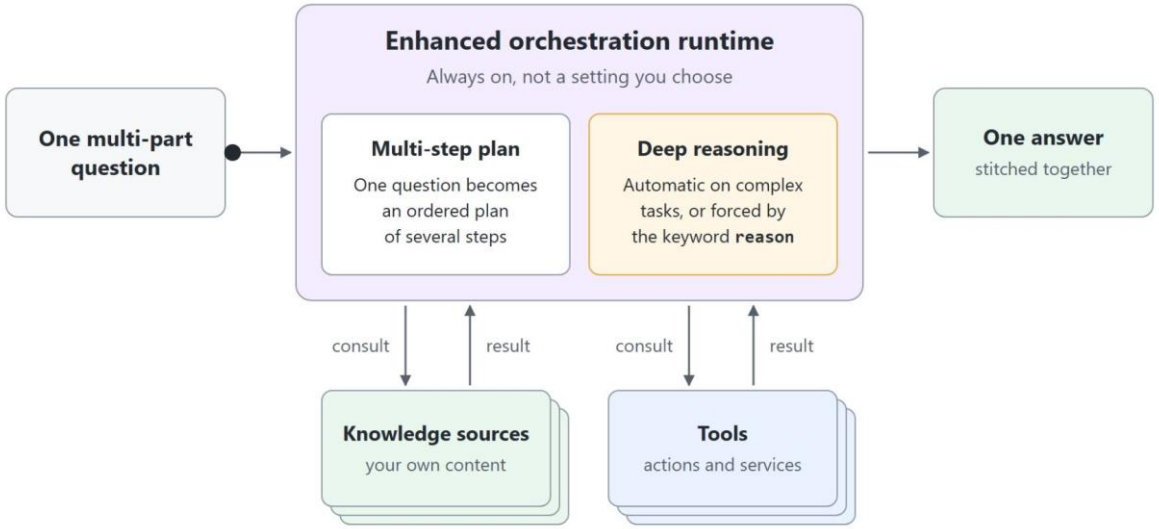
The Unified Build Surface and the New Orchestrator

Four tabs, one components panel and an always-on runtime

The enhanced orchestration runtime

How one question becomes a plan across sources and tools

- **Build tab**: Instructions, Knowledge, Tools and Skills sit in one components panel
- **Always on**: every Microsoft Copilot harness agent runs this runtime; there is no setting to pick
- **Multi-step plans**: one question can draw on several knowledge sources and tools and come back as one answer
- **Deep reasoning**: applied on its own to complex tasks, or forced with the keyword reason in an instruction
- **Visible plan**: the step list in Preview shows each step, and History keeps every Preview session

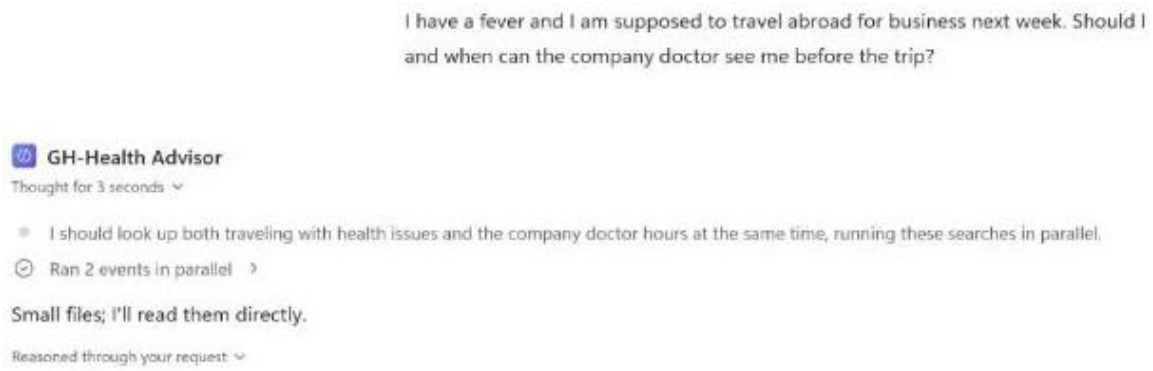


Reading the plan in Preview

A reason instruction turns one question into parallel searches

- With **Debug mode** on, the step list shows the plan, then **Ran 2 events in parallel**: one knowledge search per part of the question

```
When an employee mentions a trip and a symptom,
reason step by step:
first check whether the symptom allows travel,
then look up the
company doctor's consultation hours and booking
rules, then summarize.
```



03

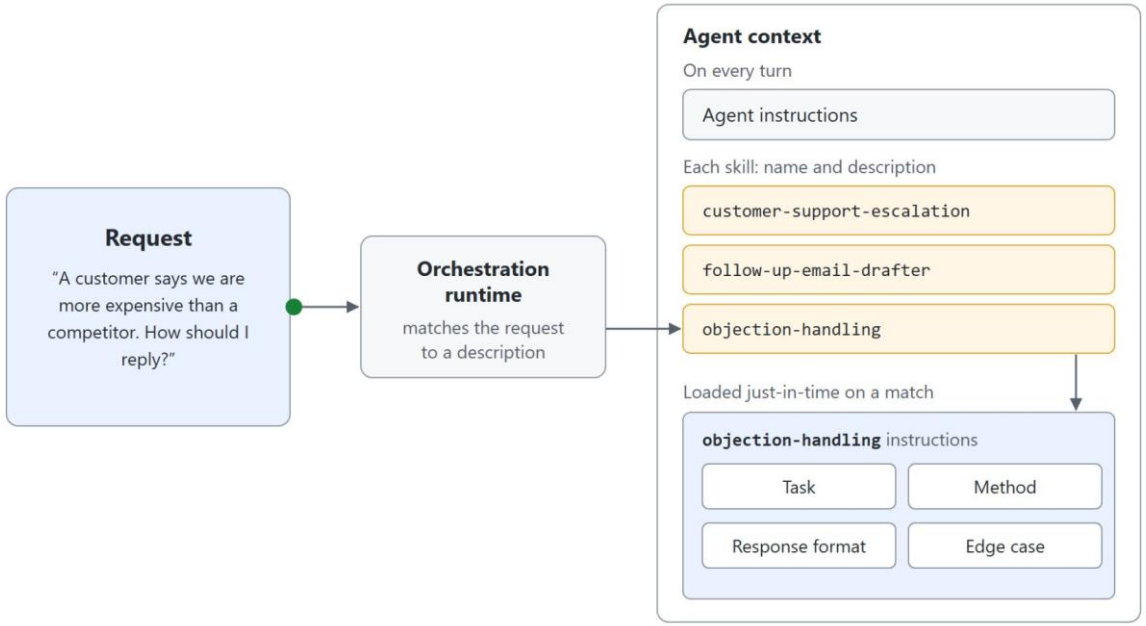
Agent Skills in Copilot Studio

SKILL.md recipes that load when a request matches

Agent skills

A SKILL.md recipe that loads when a request matches its description

- **SKILL.md**: YAML front matter with a name and a description, then Markdown instructions
- **On demand**: only names and descriptions stay in context; the full instructions load on a match
- **Description**: the runtime activates a skill from it alone, so it carries the caller's words
- **Portable**: the same open format as Claude Code and GitHub Copilot, uploaded or downloaded in the Skills panel
- **With tools**: a skill has no live connection of its own, but it can direct the agent to use a tool

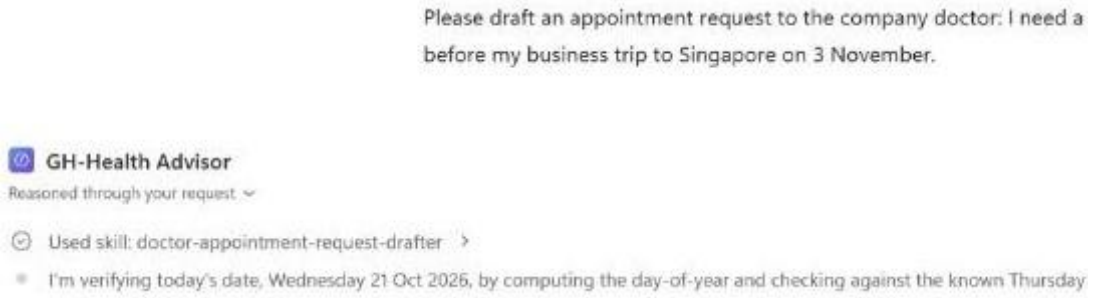


Testing a skill in Preview

The description decides when the skill fires

- **Used skill** in the step list proves the match; a near-miss question must stay quiet

```
---
name: doctor-appointment-request-drafter
description: Draft an appointment request to the company doctor
  ONLY when the employee explicitly asks to book, request, or
  schedule an appointment, a pre-travel check, or a fit-to-travel
  note. Do not use for general health questions, symptom advice,
  fasting or nutrition questions, travel advice, or questions
  about consultation hours.
---
```



04

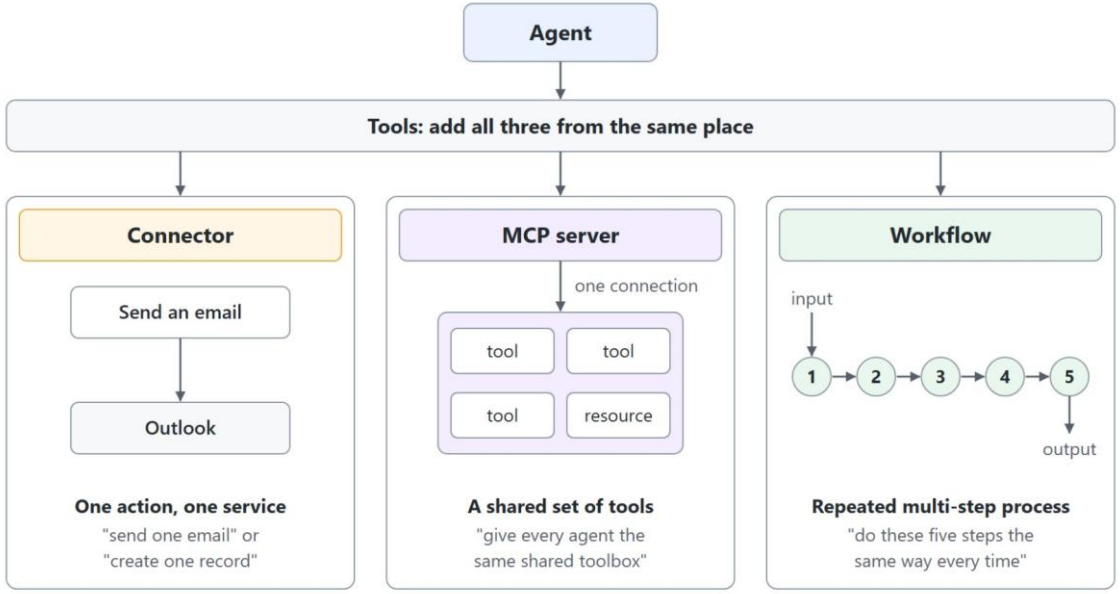
Tools, MCP, and the New Workflows Designer

Connectors, MCP servers and workflows with agent nodes

Connectors, MCP servers and workflows

Which tool fits which job

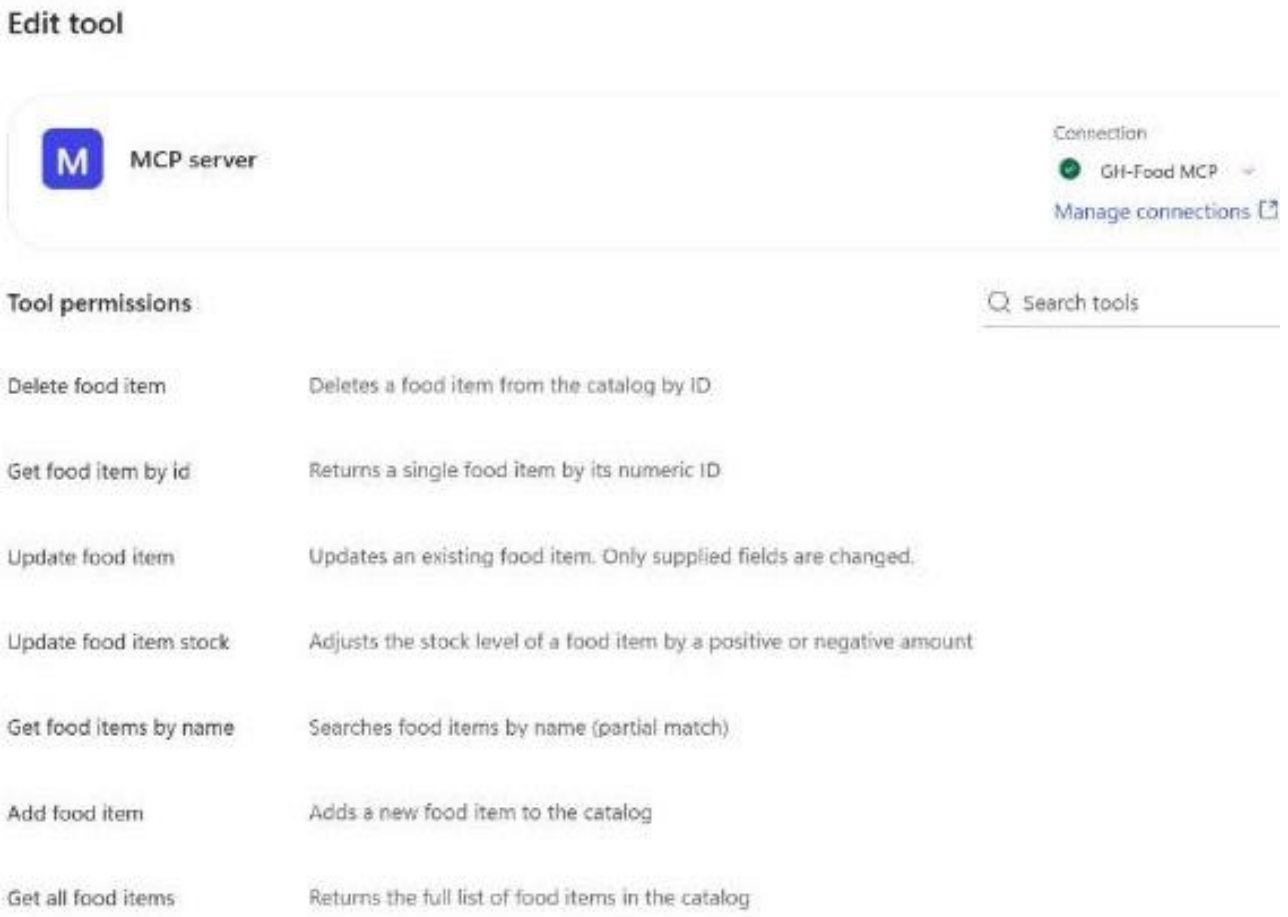
- **Connector**: one action against one service, such as sending one email
- **MCP server**: one connection to a shared set of tools; its description tells the orchestrator when to call it
- **Workflow**: a repeated multi-step process, tested node by node and versioned in the new designer
- **Agent node**: a workflow step that hands a task to your agent and passes the result on



Adding an MCP server

One connection, every tool the server exposes

- **Add a tool**, MCP tab for a prebuilt server, or **Model Context Protocol** for your own
- Server name, **description**, URL and authentication: None, API key or OAuth 2.0
- **Edit tool** lists every tool the server exposes under **Tool permissions**
- The orchestrator calls the server on its own when a question matches the description



A workflow with an agent node

Fixed steps around one call to your agent



- Trigger, **Agent** node and **Respond to the agent** ran in 17 seconds; the agent node called the published GH-Food Buddy

05

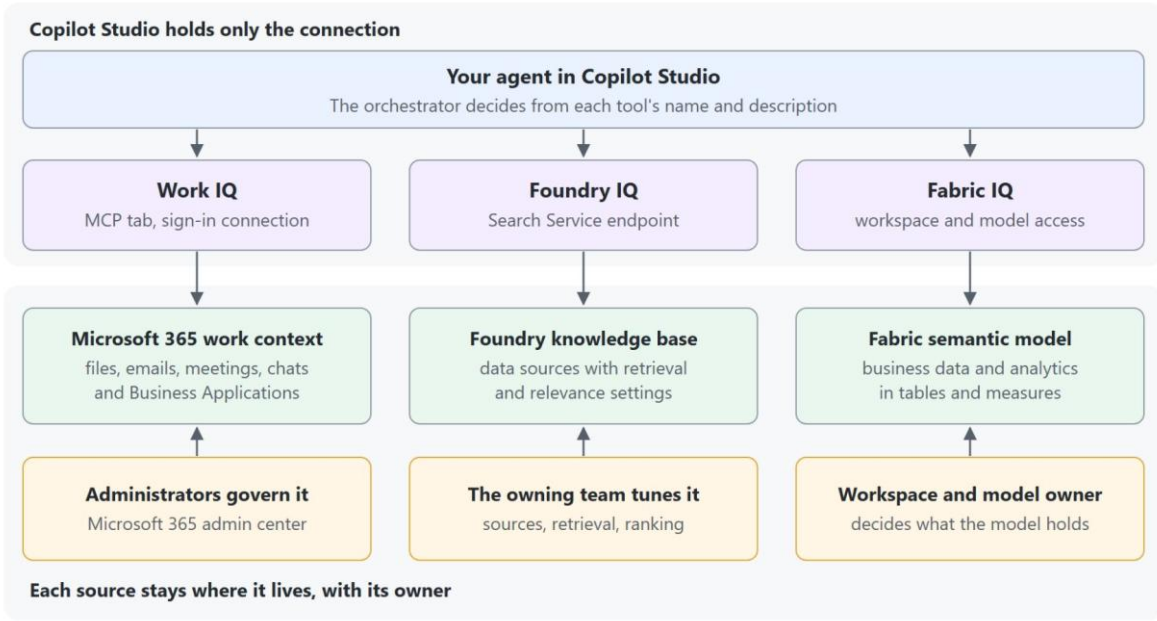
Grounding Agents with Work IQ, Foundry IQ, and Fabric IQ

Sources that stay where they live, proven in the trace

Grounding with Work IQ, Foundry IQ and Fabric IQ

The agent reads each source where it lives

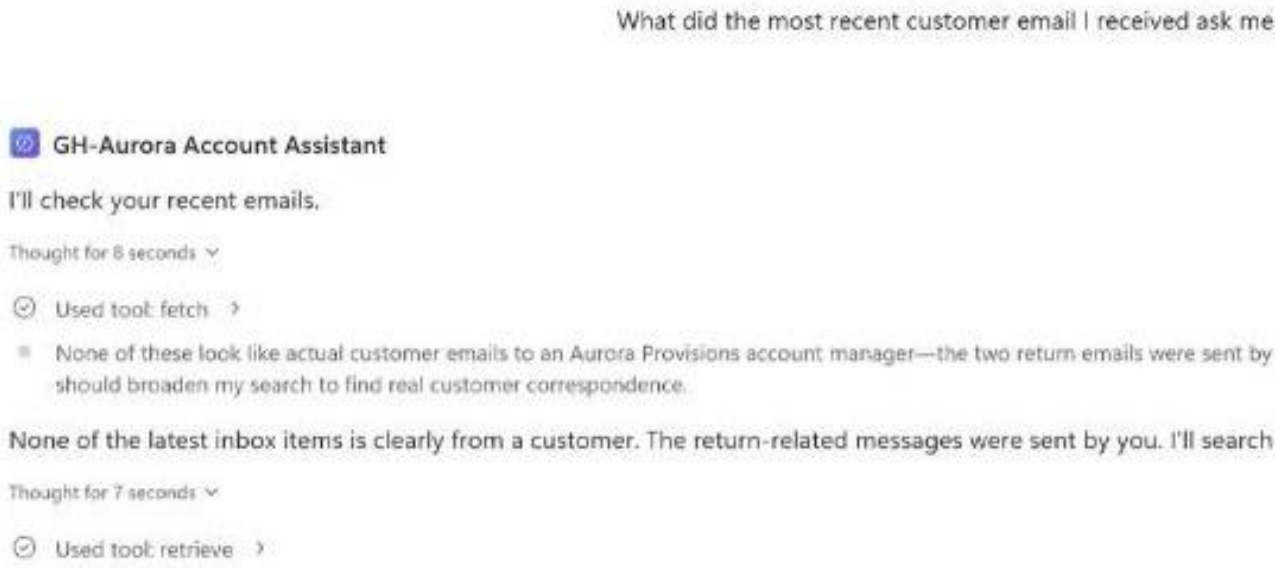
- **Work IQ** reads your Microsoft 365 email, meetings, chats and files
- **Foundry IQ** answers over a knowledge base a team tunes in Microsoft Foundry
- **Fabric IQ** answers data questions from a semantic model in Microsoft Fabric
- Copilot Studio holds only the **connection**; each source stays with its owner



Proving grounding in the activity trace

The same question, asked again after Work IQ is added

- Add **Work IQ (Preview)** from the MCP tab of **Add a tool**, not the legacy Mail or Teams tools
- Ask the **same question** before and after: only the tool changed
- The trace shows **Used tool: retrieve** and **fetch**; expand a line for its input and output
- **Debug mode** must be on to expand the lines



Live sources and uploaded files

Why grounding matters for information that moves

- An **uploaded file** is an indexed copy: the agent knows what you uploaded last
- An **IQ connection** reads the source where it lives when the question arrives
- That matters for information that moves, such as a **customer email from this morning**
- You change a file by uploading a new version; an IQ source is tuned by its **owner**



06

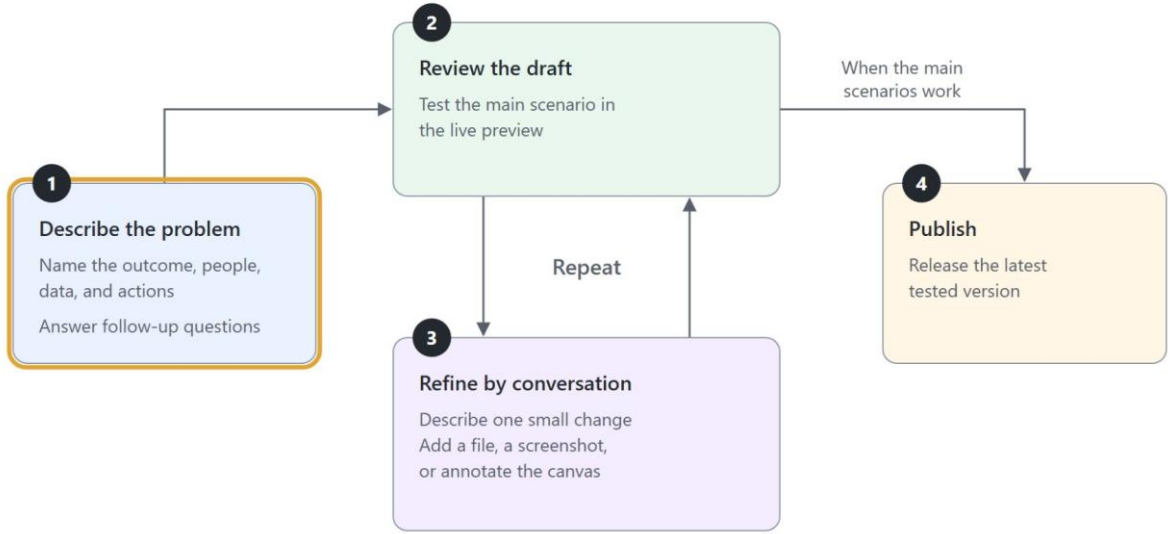
Building Apps in Copilot Studio

Describe, refine by conversation, publish and share

Building apps

Describe the app, then review and refine it until the main scenarios work

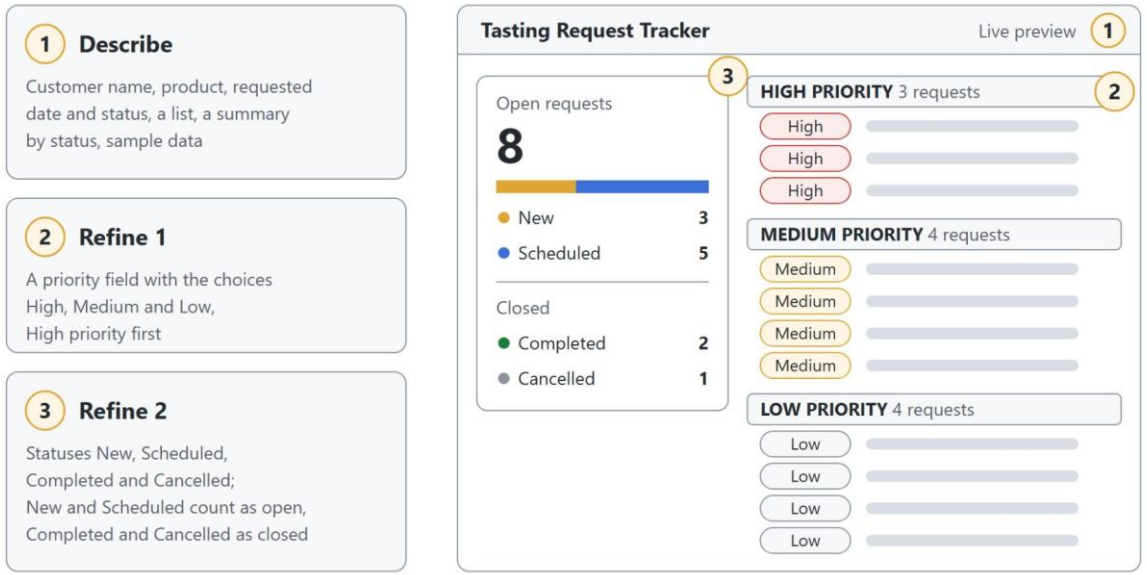
- **Describe**: name the outcome, people, data and actions; Copilot Studio generates the data model, logic and interface
- **Refine**: one small change per message, checked in the live preview; the code view is read-only
- **Data access**: granted on the SharePoint list or Dataverse table, separately from sharing the app
- **Copilot Credits**: billing starts as soon as you start building



The Tasting Request Tracker

An app refined by conversation, checked in the live preview

- **Describe**: one message named the people, the data and the actions; the first draft took about five minutes
- **Refine 1**: a priority field, with High requests listed first
- **Refine 2**: four statuses, with New and Scheduled counted as open
- **Check** each change in the preview before sending the next one



07

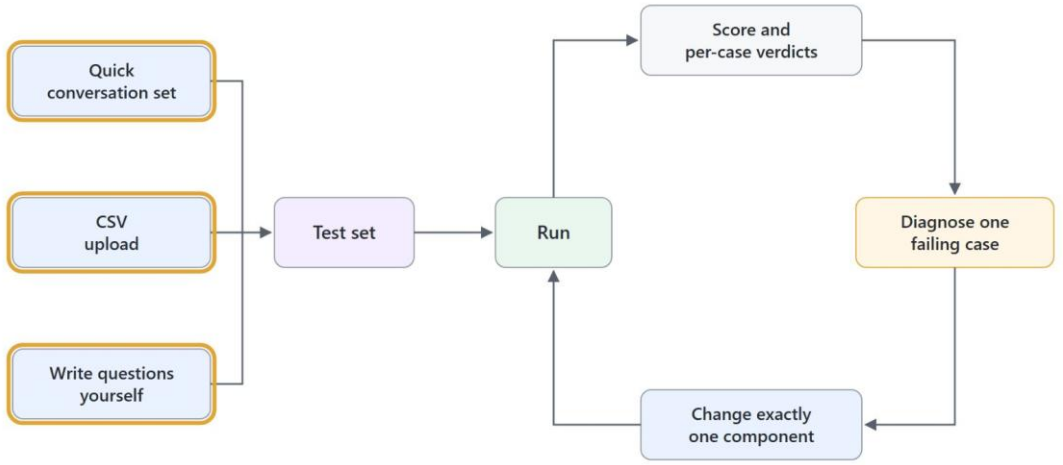
Test and Evaluate Copilot Studio Agents

Test sets, General quality and a failing case to its fix

Evaluating agents with test sets

Fill a set, run it, change one thing, run it again

- **Test set**: conversations of user questions with optional expected responses, filled by Quick conversation set, CSV upload or by hand
- **General quality**: the one test method on the Evaluate tab; it judges relevance and completeness and does not compare to the expected response
- **User profile**: the identity a run executes as, matched to the access real users have
- **Invalid and Error**: about the run or the case, so clear them before reading Pass and Fail
- **One change per run**: baseline, one change, re-run the same set, compare the cases that flipped



A test set worth trusting

Weight the cases toward where the agent is likely to fail

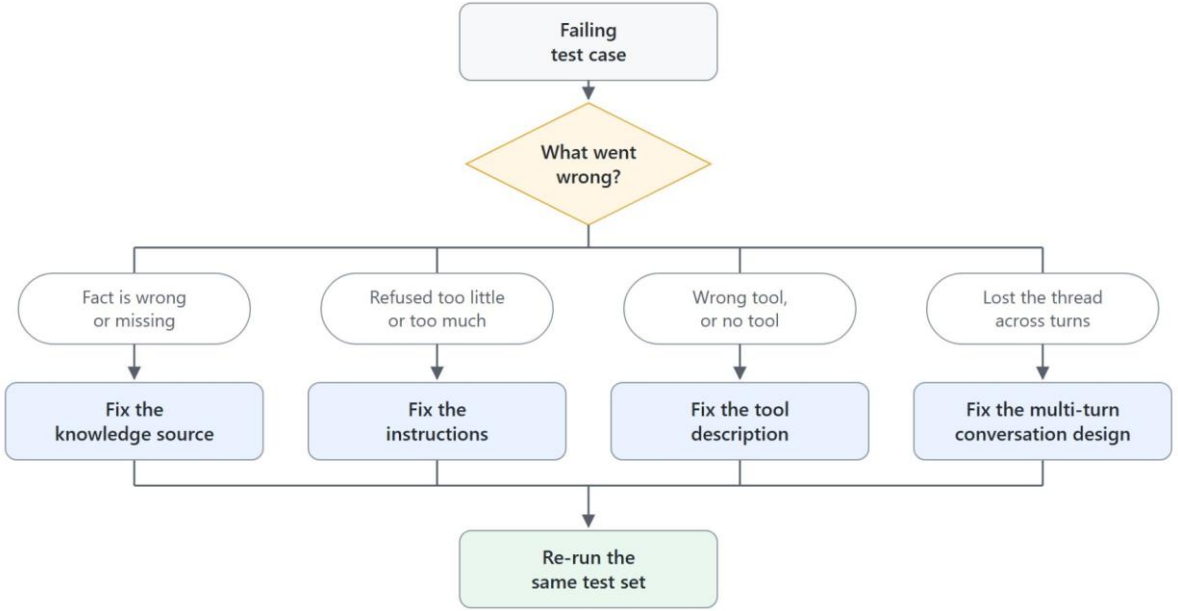
- **Grounded facts**: the agent reports something from its knowledge correctly
- **Guardrails**: it refuses what it should, such as a binding price quote
- **Off-topic**: it declines instead of answering from general model knowledge
- **Multi-turn**: it holds context and asks for the missing detail



From a failing case to a fix

What went wrong names the component to change

- **Wrong or missing fact**: fix the knowledge source
- **Refused too little or too much**: fix the instructions
- **Wrong tool or no tool**: fix the tool description
- **Lost the thread across turns**: fix the multi-turn design, then re-run the same set



Takeaways

What to carry into publishing and maintaining

- Decide the **kind of agent** before you build: there is no conversion
- Read the **step list** and **History** to see how the orchestrator planned
- A skill's **description** and an MCP server's **description** decide when they are used
- Prove grounding in the **activity trace**, not in the wording of the reply
- Run the **same test set** before and after every change

